

ECOM EXPERTS

AN Product Fields 8.2

Manual de usuario e instalacion

an_productfields82 2.5.0 · 04/09/2026

Contenido

1. Qué hace y para quién	
2. Compatibilidad	
3. Instalación	
4. Configuración	
4.1 La lista de campos	
4.2 Ajustes generales	
4.3 Crear o modificar un campo	
5. Cómo se usa, día a día	
Lo que hay que revisar cada vez que se crea un campo	
6. Qué ve el cliente en la tienda	
7. Preguntas frecuentes y problemas	
8. Seguridad	
Fallos de seguridad encontrados y CORREGIDOS	
Errores que tumbaban la tienda en PrestaShop 9, corregidos	
Otros arreglos	
Revisado y correcto, sin cambios	
Pendiente (no se ha tocado: cambiaría el comportamiento del módulo)	
9. Ficha técnica	

1. Qué hace y para quién

Este módulo añade **campos que rellena el cliente** en la ficha de producto, justo encima del botón de añadir al carrito. Sirve para todo lo que hay que preguntarle al comprador antes de fabricar o preparar el pedido: el texto de un grabado, la talla de un anillo, la fecha de una entrega, el color de un ribete, el nombre que va bordado en una toalla, el mensaje de una tarjeta de regalo o el archivo de un diseño.

Cada campo puede llevar **precio propio**. Si el cliente rellena «Texto grabado», el módulo suma ese importe al precio del artículo en el carrito, en el pedido, en el correo de confirmación y en la factura. El comercio no tiene que crear combinaciones ni productos duplicados para cobrar un extra: se define el campo una vez y se aplica a todo el catálogo, a unas categorías concretas o a los productos que se elijan uno a uno.

Los campos admiten siete formatos —texto, área de texto, fecha, botones de opción, lista desplegable, selección múltiple y casilla— y se pueden validar (solo números, solo letras, correo electrónico, dirección web o una lista de caracteres que decida el comercio). Cada opción de una lista puede tener a su vez su propio precio y hasta su propia imagen, de manera que el cliente elige «acabado mate (+2 €)» o «acabado brillante (+5 €)» y el importe se ajusta solo, en pantalla, sin recargar la página.

Está pensado para tiendas que venden **producto personalizado**: joyería, regalos, rotulación, textil, imprenta, alimentación con dedicatoria, mobiliario a medida. En la práctica sustituye a la personalización nativa de PrestaShop, que no sabe cobrar por el dato introducido ni ofrecer listas de opciones con precio.

2. Compatibilidad

PrestaShop	8.0 – 8.2 (objetivo del módulo). Probado además en 9.2.0-beta.1 , con los arreglos descritos en el apartado 8
PHP	7.4 y 8.0 – 8.2
Temas	Diseñado para classic y temas derivados de él. Con hummingbird (el tema por defecto de PrestaShop 9) hay una limitación en el carrito: ver el apartado 7
Multitienda	Sí. Cada campo se asocia a las tiendas que se elijan (tabla <code>an_productfields_shop</code>) y las opciones de configuración se guardan por contexto de tienda
Idiomas	El nombre, el marcador de posición y los valores de cada campo son multiidioma . Los textos del back-office están en inglés (ver apartado 7)
Dependencias	Ninguna. Sin Composer, sin librerías externas, sin servicios de terceros
Convive con <code>an_productfields</code>	Sí. Es la misma base de datos: las dos copias leen y escriben las mismas tablas, así que se puede desactivar una y activar la otra sin perder nada

Este módulo instala **un override** de la clase `Cart` (`override/classes/Cart.php`). Es imprescindible: es lo que suma el precio de los campos al carrito y al pedido.

3. Instalación

- Subir el ZIP desde **Módulos** → **Gestor de módulos** → **Subir un módulo**, o descomprimir la carpeta `an_productfields82` dentro de `modules/`.
- Pulsar **Instalar**.
- Al terminar aparece un menú nuevo: **Catálogo** → **AN Product Fields**.

Qué crea la instalación

- Siete tablas: `an_productfields`, `an_productfields_lang`, `an_productfields_values`, `an_productfields_shop`, `an_productfields_product`, `an_productfields_cart` y `an_productfields_cart_values` (todas con el prefijo de la tienda).
- Una pestaña de administración, `AdminAnProductFields82`, colgando de Catálogo.

- Dieciséis hooks (lista completa en el apartado 9).
- El fichero `override/classes/Cart.php`. Si ya existía uno del módulo `an_productfields` original, se guarda una copia en `Cart.php.an_pf_backup` antes de sustituirlo. Si existía uno de otro módulo, **no se toca** y la pantalla del módulo avisa en rojo.

Qué pasa al desinstalar

- Se quita la pestaña, se dan de baja los hooks y **se retira el override** (si había copia de seguridad, se repone la original).
- **Las tablas NO se borran.** Es a propósito: son las mismas que usa `an_productfields`, y borrarlas dejaría al otro módulo sin datos. Para borrarlas de verdad hay que pedirlo expresamente con el parámetro `an_pf_drop_tables` en la desinstalación.

Al actualizar el módulo no hace falta hacer nada: al entrar en su pantalla se ejecuta `addnewfeatures()`, que añade las columnas que falten, da de alta los hooks nuevos y repone el override si se ha quedado atrás.

4. Configuración

La pantalla del módulo está en **Catálogo → AN Product Fields** y tiene dos pestañas propias: *Product fields* (la lista de campos) y *Settings* (los ajustes generales).

4.1 La lista de campos

PRESTASHOP

9.2.0

Acceso rápido

Buscar

Modo depuración

Ver mi tienda

Inicio

VENDER

Pedidos

Catálogo

Productos

Categorías

Monitoreo

Atributos y Características

Marcas y Proveedores

Archivos

Descuentos

Stock

Ebooks Narcea

Venta por medidas

Ofertas de unidades extra

Unidades extra: consentimientos

AN Product Fields

Cientes

Servicio al Cliente

Estadísticas

Presupuestos

PERSONALIZAR

Módulos

Diseño

Transporte

Pago

Internacional

CONFIGURAR

Parámetros de la tienda

Parámetros Avanzados

COMUNIDAD

Muro de la fama

MIRAKL LIST

Mirakl Orders List

Mirakl Prices List

Mirakl Exclude Products

Mirakl Promos By category

Mirakl Promos By Brand

Mirakl Logística por categoría

Mirakl Logística por Producto

Aumento por Rango de precios

Mapeo transportes MK

Mapeo de Transportistas para Shipping

Mirakl Prices By EAN

Aumento por Categoría

Aumento por Marca

Cats By Mirakl

Mirakl - Paises

Mirakl - Productos activos

Mirakl Promos By EAN

Mirakl Campos personalizados

Mirakl - Asociar Shops a Market

Mirakl - Excluir por Supplier

Mirakl - GPRS Marcas

Mirakl - GPRS Products

Catálogo > AN Product Fields

AN Product Fields

Ayuda

Product fields

Settings

Support

Another modules

Rating:

If you like this module, Please RATE me 5 stars [here](#).

Thank you very much!

AN Product Fields

ID

Internal name

Price

Type

Sort Order

Date Add

Date Last Update

Enabled

1

grabado

3,50 €

text

1

2026-09-04 00:35:36

2026-09-04 00:35:36

Modificar

Ecom Experts

Pagina 6 de 21



Aquí están todos los campos creados, con su nombre interno, su precio, su tipo, el orden en que salen y si están activos. El botón  de la esquina crea uno nuevo; el desplegable de cada fila permite modificarlo o borrarlo.

4.2 Ajustes generales

PRESTASHOP 9.2.0 Acceso rápido 🔍 Buscar

Modo depuración Ver mi tienda Ayuda

Inicio

VENDER

Pedidos

Catálogo

Productos

Categorías

Monitoreo

Atributos y Características

Marcas y Proveedores

Archivos

Descuentos

Stock

Ebooks Narcea

Venta por medidas

Ofertas de unidades extra

Unidades extra: consentimientos

AN Product Fields

Cientes

Servicio al Cliente

Estadísticas

Presupuestos

PERSONALIZAR

Módulos

Diseño

Transporte

Pago

Internacional

CONFIGURAR

Parámetros de la tienda

Parámetros Avanzados

COMUNIDAD

Muro de la fama

MIRAKL LIST

Mirakl Orders List

Mirakl Prices List

Mirakl Exclude Products

Mirakl Promos By category

Mirakl Promos By Brand

Mirakl Logística por categoría

Mirakl Logística por Producto

Aumento por Rango de precios

Mapeo transportes MK

Mapeo de Transportistas para Shipping

Mirakl Prices By EAN

Aumento por Categoría

Aumento por Marca

Cats By Mirakl

Mirakl - Países

Mirakl - Productos activos

Mirakl Promos By EAN

Mirakl Campos personalizados

Mirakl - Asociar Shops a Market

Mirakl - Excluir por Supplier

Mirakl - GPRS Marcas

Mirakl - GPRS Products

Catálogo > AN Product Fields

AN Product Fields

Product fields

Settings

Support

Another modules

Rating:

If you like this module, Please RATE me 5 stars [here](#).

Thank you very much!

Expert Settings

Enable display in a separate table

Enabled

Apply tax to fields value

Enabled

Apply specific price to fields value

Enabled

Productfields header text

Additional information

Leave this field empty if you don't want the block with productfields to have a header.

Productfields footer text

Leave this field empty if you don't want the block with productfields to have a footer.

Product added to cart modal

Original

Simple

None

This option does not work if your theme override's "ps_shoppingcart/modal.tpl"

Dynamic product price change for storefront

Enabled

Display fields prices for storefront

Enabled

Display symbol counter for text fields

Enabled

HTML tag for price

The option is used in case when your theme was changed a lot. It's recommended not to use it without instructions of the developer.

HTML tag for product fields

.product-add-to-cart

The option is used in case when your theme was changed a lot. It's recommended not to use it without instructions of the developer.

JS position for product fields

Before

After

Prepend

Append

The option is used in case when your theme was changed a lot. It's recommended not to use it without instructions of the developer.

Custom validator

ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789_-

Enter the characters to be successfully validated.

Corregir el precio en los correos del pedido

Enabled

Hace que el correo de confirmación muestre el precio con los campos sumados. En PrestaShop 8.2 ya no hace falta sobrescribir PaymentModule: se resuelve con el hook actionPaymentModuleProductVarTplAfter.

Clear Prestashop Cache

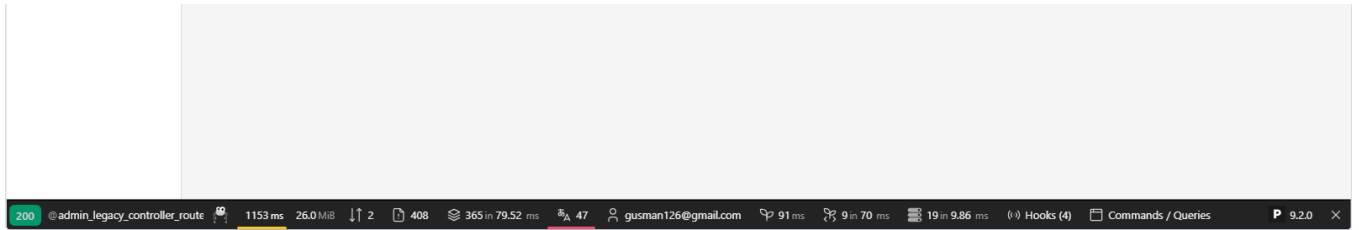
Disabled

Soft reset module

Disabled

If a different theme is selected, you must restart the module with the same parameters

Save



Qué hace cada opción de verdad:

Opción	Qué provoca	Qué poner
Enable display in a separate table	Los campos se pintan en su propio bloque en lugar de mezclarse con los atributos del producto	Activado, salvo que el tema quede raro
Apply tax to fields value	El precio del campo se muestra y se cobra con IVA . Si se desactiva, el precio que se teclea en el campo se trata como precio sin impuestos	Activado en tiendas B2C
Apply specific price to fields value	Si el producto tiene un descuento (por ejemplo -20 %), ese mismo descuento se aplica al precio del campo	Activado si el extra debe rebajarse igual que el artículo; desactivado si el grabado se cobra siempre entero
Productfields header text	Título que aparece encima del bloque de campos en la ficha	Un texto corto: «Personalízalo», «Datos del grabado». Vacío = sin título
Productfields footer text	Texto debajo del bloque	Avisos: «Revise la ortografía: los grabados no admiten devolución»
Product added to cart modal	Sustituye la ventana emergente de «producto añadido» del tema classic por una versión simple o la elimina	Dejar en <i>Original</i> salvo que la ventana del tema no muestre bien los campos. Solo funciona sobre el tema classic : sobrescribe <code>themes/classic/modules/ps_shoppingcart/modal.tpl</code>
Dynamic product price change for storefront	Al rellenar o cambiar un campo, el precio grande de la ficha se actualiza al vuelo con el extra incluido	Activado. Es lo que evita la sorpresa en el carrito
Display fields prices for storefront	Muestra el (+4,24 €) al lado del nombre del campo y de cada opción	Activado
Display symbol counter for text fields	Pinta el contador (10/40) bajo los campos de texto con longitud máxima	Activado
HTML tag for price	Selector CSS del elemento donde está el precio, para el cambio en vivo	Vacío. Solo se toca si el tema no usa las clases estándar
HTML tag for product fields	Selector CSS junto al que se coloca el bloque de campos	<code>.product-add-to-cart</code> (el valor por defecto y el que funciona en classic y en hummingbird)
JS position for product fields	Si el bloque va antes, después, al principio o al final de ese elemento	<i>Before</i>
Custom validator	Lista de caracteres admitidos cuando un campo usa la validación <i>Personalizar</i>	Los que permita el proceso: por ejemplo, si la grabadora no hace eñes, no ponerla
Corregir el precio en los correos del pedido	El correo de confirmación muestra el precio con los campos sumados en lugar del precio de catálogo	Activado
Clear Prestashop Cache	Vacía la caché al guardar	Solo si un cambio no se refleja
Soft reset module	Desinstala y reinstala el módulo sin borrar los datos	Solo para reparar

Se guarda todo junto con el botón **Save**. Comprobado sobre PrestaShop 9.2: los valores se escriben de verdad en la tabla `ps_configuration`, no solo en pantalla.

4.3 Crear o modificar un campo

Pagina 14 de 21

Campo	Qué es
Nombre	Lo que ve el cliente. Es multiidioma. No admite ; : " , ni - , porque el módulo usa esos caracteres para separar valores internamente
Internal name	Identificador para el comercio. Solo letras, números, guiones y guiones bajos
Price	Lo que se suma al precio del artículo si el cliente rellena el campo. Puede ser 0
Status	Activo o no. Un campo desactivado desaparece de todas las fichas
Sort Order	Orden de aparición cuando hay varios campos en el mismo producto
Required	Si es obligatorio, no se puede añadir al carrito sin rellenarlo
Type	Texto, área de texto, fecha, botones de opción, selección múltiple, lista desplegable o casilla
Validation	Ninguna, alfanumérica, numérica, alfabética, dirección web, correo electrónico o <i>Personalizar</i> (la lista de caracteres del apartado anterior)
Placeholder	El texto gris de ejemplo dentro de la caja
Max Text Length	Máximo de caracteres. Es lo que alimenta el contador (10/40) . Vacío = sin límite
Customer Groups	Grupos de clientes que ven el campo. Obligatorio marcar al menos uno
Apply this field for all products	Activado, el campo sale en todo el catálogo
Apply to all products in selected categories	Alternativa a lo anterior: solo en las categorías marcadas
Custom tax	Regla de impuestos propia para el extra, si tributa distinto que el artículo

Cuando el tipo es lista desplegable, botones de opción o selección múltiple, al guardar aparece debajo un panel para dar de alta **los valores**, cada uno con su texto, su precio, un enlace y, opcionalmente, **una imagen**. Las imágenes se guardan en `modules/an_productfields82/views/img/` y solo se admiten JPG, PNG, GIF y WEBP.

5. Cómo se usa, día a día

Caso 1 — Un grabado que se cobra a todo el catálogo. Crear un campo de tipo *Text*, precio 3,50, longitud máxima 40, marcar *Apply this field for all products* y guardar. Desde ese momento, toda ficha de producto muestra la caja de texto con su contador y su recargo.

Caso 2 — Un acabado con varias opciones y precios distintos. Crear un campo de tipo *Dropdown*, precio 0, guardar, y en el panel de valores añadir «Mate» con precio 0, «Brillante» con precio 2 y «Metalizado» con precio 5. El cliente elige y el precio de la ficha cambia solo.

Caso 3 — Un dato obligatorio solo para una familia de productos. Crear el campo, desmarcar *Apply this field for all products* y marcar las categorías. Poner *Required* en sí: el botón de añadir al carrito avisará si el cliente lo deja vacío.

Caso 4 — Un campo solo para clientes registrados. En *Customer Groups*, marcar únicamente «Cliente». Los visitantes anónimos no lo verán.

Lo que hay que revisar cada vez que se crea un campo

1. Abrir una ficha de producto de la tienda y comprobar dónde cae el bloque.
2. Rellenarlo y mirar que el precio grande cambie.
3. Añadirlo al carrito y comprobar que el dato y el recargo aparecen en la línea.

6. Qué ve el cliente en la tienda

En la ficha de producto, encima del botón de añadir al carrito:

Contacte con nosotros
Iniciar sesión
Carrito 0

my store
Ropa
Accesorios
Arte

Buscar productos...

Inicio / Ropa / Hombre / Hummingbird printed t-shirt

-20%
Nuevo



Hummingbird printed t-shirt

Studio Design

28,92 € (20% de descuento)

23,14 €

Impuestos incluidos

Regular fit, round neckline, short sleeves. Made of extra long staple pima cotton.

Tamaño: S

S

Color: Blanco

☐ ☐

Additional information

Texto grabado (+4,24 €)

PARA MARIA

(10/40)

- 1 +

Añadir al carrito

Compartir

Este producto no admite devolución

Este producto no admite devolución ni desistimiento por tratarse de un artículo personalizado o excluido por ley (art. 103 TRLGDCU). Revise bien su elección antes de comprar.

Este artículo se fabrica en lotes de 8 unidades

Puedes llevarte 7 unidades más con un 50% de descuento

Este artículo se fabrica de 8 en 8 unidades. Como has pedido 1, en el taller sobrarían 7 unidades que acabarían en la basura.

SI ACEPTAS:		SI ACEPTAS
	AHORA	
Unidades que recibes	1	8 (+7)
Precio del artículo	23,14 €	104,11 € (+80,97 €)
Te sale a	23,14 € / ud	13,01 € / ud

Si aceptas, añadimos esas 7 unidades a tu pedido: recibirás 8 en lugar de 1 y pagarás 80,97 € más (11,57 € por unidad en vez de 23,14 €). Tu pedido pasa de 23,14 € a 104,11 €.

Se ve el título del bloque («Additional information», configurable), el nombre del campo con su recargo entre paréntesis, la caja con su texto de ejemplo y el contador de caracteres. Al escribir, el precio del artículo se recalcula sin recargar la página.

En el carrito, el dato y su importe aparecen bajo el nombre del producto, igual que si fueran atributos, y el total de la línea ya lleva el extra sumado:

Contacte con nosotros

Iniciar sesión

Carrito 1

my store

Ropa ▾ Accesorios ▾ Arte

Q

Buscar productos...

Inicio / Carrito

Carrito



Hummingbird printed t-shirt

Tamaño: S

Color: Blanco

Texto grabado (+3.39 €): PARA MARIA

26,52 €

33,15 € -20%

Eliminar

-

1

+

26,52 €

← Continuar comprando

Resumen del pedido

Hummingbird printed t-shirt: puedes llevarte 7 unidades más con un 50% de descuento

Este artículo se fabrica de 8 en 8 unidades. Como has pedido 1, en el taller sobrarían 7 unidades que acabarían en la basura.

SI ACEPTAS:

	AHORA	SI ACEPTAS
Unidades que recibes	1	8 (+7)
Precio del artículo	23,14 €	104,11 € (+80,97 €)
Te sale a	23,14 € / ud	13,01 € / ud

Si aceptas, añadimos esas 7 unidades a tu pedido: recibirás 8 en lugar de 1 y pagarás 80,97 € más (11,57 € por unidad en vez de 23,14 €). Tu pedido pasa de 23,14 € a 104,11 €.

Te salen a 13,01 € por unidad en lugar de 23,14 €.

Las unidades extra se fabrican expresamente para este pedido y no admiten devolución por desistimiento. Si no aceptas, recibirás las 1 unidades que pediste.

→ Al aumentar las unidades puede variar el coste de envío.

☐ Si, quiero las 7 unidades extra por 80,97 € más. He leído las condiciones.

Añadir las 7 unidades extra

No, gracias

1 artículo

26,53 €

Descuento(s)

-23,14 €

Transporte

Gratis

Total (impuestos inc.)

3,39 €

🛒

Descuento por unidades extra de fabricación (3 uds al 50%)

-26,52 €

Código promocional

▼

Finalizar compra

🔒

Security policy

(edit with the Customer Reassurance module)

🚚

Delivery policy

(edit with the Customer Reassurance module)

🔄

Return policy

(edit with the Customer Reassurance module)

En la captura, la camiseta cuesta 23,14 € y el grabado 3,39 € (3,50 € menos el 20 % de descuento del artículo, con IVA): la línea suma 26,52 €.

Lo mismo se ve después en el detalle del pedido del cliente, en la ficha del pedido del back-office, en el correo de confirmación y en la factura en PDF.

7. Preguntas frecuentes y problemas

Los campos no se suman al total del carrito. Falta el override. La propia pantalla del módulo lo detecta y avisa en rojo: «El override del carrito (override/classes/Cart.php) NO está instalado». Se repone entrando en la pantalla del módulo (se reinstala solo) o revisando los permisos de escritura de `override/classes/`.

Tengo instalado también `an_productfields` y al desactivar uno se rompe el otro. Es un caso previsto. Los dos módulos comparten tablas y comparten override; el de esta copia lleva una marca (`AN_PRODUCTFIELDS82_OVERRIDE`) y se repone solo al abrir el back-office si otro módulo se lo ha llevado por delante.

Ecom Experts

Pagina 17 de 21

En el carrito, el botón + no cobra el grabado de la unidad nueva. Ocurre con el tema **hummingbird** (el que trae PrestaShop 9 de serie), no con classic. Los botones de cantidad de hummingbird no envían el identificador interno del campo, así que: el + añade una unidad **sin personalizar** como línea aparte, y el - o el enlace *Eliminar* borran la línea de producto entera con sus campos. **Está descrito en el apartado 8 como pendiente.** Mientras tanto, en tiendas con hummingbird conviene que el cliente ajuste la cantidad en la ficha de producto antes de añadir al carrito.

He cambiado una opción y la tienda sigue igual. Es la caché. Activar *Clear Prestashop Cache* al guardar, o vaciar la caché desde Parámetros avanzados → Rendimiento.

El bloque de campos sale en un sitio raro del tema. Cambiar *HTML tag for product fields* por un selector CSS que exista en el tema (por ejemplo `.product-actions` o `.product-prices`) y ajustar *JS position*.

Los textos del back-office están en inglés. El módulo usa el sistema de traducción antiguo de PrestaShop y su fichero `translations/es.php` viene vacío. Se pueden traducir desde **Internacional** → **Traducciones** → **Traducciones de módulos**, o dejarlo así: solo lo ve el comercio, no el cliente. Lo que ve el cliente (nombres de campo, marcadores, valores) es multiidioma y se escribe en el formulario.

Se me han borrado las imágenes de los valores al actualizar el módulo. Se guardan dentro de la carpeta del módulo (`views/img/`), y subir un ZIP nuevo la reemplaza. Hacer copia de esa carpeta antes de actualizar.

¿Puedo dejar que el cliente suba un archivo? No: los tipos disponibles son texto, área de texto, fecha, opciones, lista, selección múltiple y casilla. Las imágenes las sube el comercio, para ilustrar las opciones.

8. Seguridad

Revisión completa del 4 de septiembre de 2026 sobre PrestaShop 9.2.0-beta.1: fichero principal, controlador de administración, controlador de tienda, clases, override del carrito, plantillas y JavaScript.

Fallos de seguridad encontrados y CORREGIDOS

1. Cualquiera podía leer los datos de cualquier pedido — `controllers/front/opc.php`, `getOrderDetailsAction()`. El endpoint aceptaba un `id_order` cualquiera y devolvía los campos personalizados de ese pedido, sin comprobar sesión ni propiedad. Un visitante anónimo recorriendo `id_order = 1, 2, 3...` se llevaba los datos de todos los pedidos de la tienda. Ahora exige cliente identificado y que el pedido sea suyo. *Comprobado: desde una sesión anónima devuelve una respuesta vacía.*

2. Se podía manipular el carrito de otro cliente — mismo fichero, `removeByHashAction()` y `updateQtyByHashAction()`. Las dos acciones borraban o cambiaban filas a partir de un identificador (`values_hash`) recibido del navegador, **sin comprobar que fuera del carrito de quien llamaba**. Ese identificador es `md5(id_campo_idcarrito_idproducto_idcombinacion_valor_precio)`: como los identificadores de carrito son correlativos, se puede calcular el de otra persona. Ahora se comprueba contra `an_productfields_cart` que el hash pertenece al carrito actual. *Comprobado: desde una segunda sesión las filas quedan intactas; desde la sesión dueña, se borran.*

3. Subida de ficheros sin ninguna comprobación — `controllers/admin/AdminAnProductFields82Controller.php`, `ajaxProcessAddValue()`. El fichero de `$_FILES` se movía a `views/img/` **con el nombre que mandaba el navegador y sin mirar la extensión**: bastaba subir un `.php` para tener ejecución de código en la tienda, y un nombre con `../` para escribir fuera de la carpeta. Ahora se comprueba `is_uploaded_file`, lista blanca de extensiones (jpg, jpeg, png, gif, webp), que el contenido sea una imagen de verdad (`getimagesize`), un tope de 4 MB, y **el nombre se genera entero** (no se conserva ni un carácter del original).

4. La carpeta de subidas era ejecutable — añadido `views/img/.htaccess` que corta el manejador de PHP y deniega el acceso a cualquier script.

5. Borrado de fichero con la ruta sin sanear — `ajaxProcessDeleteValue()` usaba el nombre guardado en la base de datos tal cual. Se le ha puesto `basename()` por si hay filas de una versión anterior del módulo, que no lo saneaba.

6. XSS almacenado entre empleados — `views/templates/admin/fields_values.tpl`. En el back-office Smarty **no** escapa solo (solo lo hace en la tienda), y ahí se pintaban sin escapar el valor, el enlace, el nombre del fichero y el idioma de cada opción. Escapados. También se ha escapado el enlace con token del back-office en `views/templates/admin/script.tpl`.

Errores que tumbaban la tienda en PrestaShop 9, corregidos

- `override_v8/classes/Cart.php`: `getProducts()` declaraba cinco parámetros y PrestaShop 9 añadió un sexto (`$shouldSplitGiftProductsQuantity`). PHP abortaba al cargar la clase y **la tienda entera se caía con un fatal, portada incluida**. Ahora la firma vale para las dos versiones y solo se le pasa el sexto al padre cuando existe.
- `override_v8/classes/Cart.php`: la constante `Cart::ONLY_PRODUCTS_WITHOUT_SHIPPING` está borrada en PrestaShop 9 y se nombraba a pelo → *Undefined constant*, otro fatal en la portada. Envuelta en `defined()`.
- `Tools::displayPrice()` ya no existe en PrestaShop 9 y se usaba en cinco sitios (plantilla de la ficha, plantilla del back-office, cálculo de precios y el endpoint AJAX). **La ficha de producto moría con un fatal** y el precio dinámico dejaba de actualizarse sin dar ni un error visible. Se ha añadido `an_productfields82::anPfDisplayPrice()`, que usa el Locale del contexto —presente en 8.2 y en 9— y cae en el método antiguo si hiciera falta.
- `AdminController::l()` tampoco existe en PrestaShop 9, y el controlador la usaba 97 veces: **toda la pantalla del módulo respondía un 500**. Se ha añadido un puente en el propio controlador que traduce a través del módulo cuando el padre no tiene el método.
- `Tools::generateIndex()` no existe en PrestaShop 9: pulsar *Clear Prestashop Cache* daba un fatal. Protegido con `is_callable`.
- *Undefined array key "id_address_delivery"* (y `id_shop`, y `quantity`) se imprimían **en mitad de la página del carrito** cuando el carrito aún no tiene dirección. Protegidos con `isset()`.
- `is_null()` **en una plantilla** no es una función registrada en el Smarty de PrestaShop 9 y soltaba un aviso en cada ficha. Sustituida por `isset()`.
- `Hook displayProductButtons`, deprecado en PrestaShop 9, imprimía un aviso *en la ficha de producto, delante del cliente*. Es un **alias** de `displayProductAdditionalInfo` (misma fila `ps_hook_alias` en 8.2 y en 9.2, o sea, el bloque sale exactamente en el mismo sitio), así que se ha cambiado por el nuevo y se da de baja el antiguo al actualizar, para que el bloque no se pinte dos veces.
- **Diez propiedades creadas al vuelo** en la clase del módulo, en el controlador de administración, en el modelo y en el override: PHP 8.2 soltaba *Creation of dynamic property* en cada petición, tanto en la tienda como en la consola. Declaradas.
- `(array)` **sobre un dato del navegador** en `hookActionProductUpdate`. Sustituido por `is_array()` con guarda por elemento.

Otros arreglos

- `installOwnOverrides()` **no reponía nunca un override que ya llevara su marca**, así que **una actualización del módulo dejaba en producción la versión antigua del fichero para siempre** — es exactamente como llegó a la tienda la firma incompatible con PrestaShop 9. Ahora compara el contenido y lo repone si difiere.
- **La marca interna** `productfields_hash: 3f2a...` **se le mostraba al cliente** en la línea del carrito con cualquier tema que no fuera classic (el JavaScript la buscaba por una clase CSS que solo existe en classic). Ahora se localiza por el texto y se oculta en cualquier tema.
- Retirado un `console.log` de depuración de `views/js/front/order.js`.

Revisado y correcto, sin cambios

- **Inyección SQL**: todas las consultas usan `(int)` o `pSQL()`. No hay `ORDER BY` ni `LIMIT` contruidos con datos del usuario.
- **Ejecución de código**: no hay `eval`, `exec`, `shell_exec`, `system` ni `unserialize` de datos externos.
- **Secretos**: no hay claves ni contraseñas en el código. El módulo no llama a ningún servicio externo.
- **Cabeceras anti-caché** del endpoint AJAX: están las tres correctas (`Cache-Control`, `Pragma`, `Expires`) y **no** manda `Content-Type: application/json`, que es lo que rompería el `JSON.parse` del JavaScript. *Comprobado sobre la respuesta real*.
- **XSS en la tienda**: las plantillas del front escapan con `escape: 'htmlall': 'UTF-8'` y además PrestaShop escapa por defecto en el front. No hay ningún `nofilter`.
- **Acciones masivas**: el módulo no recorre pedidos ni clientes históricos ni envía nada a terceros. No hay ningún riesgo de disparo masivo.
- `index.php` existe en todas las carpetas, y todos los PHP llevan `if (!defined('_PS_VERSION_')) exit;`.
- `php -l` limpio en los catorce ficheros PHP del módulo.

Pendiente (no se ha tocado: cambiaría el comportamiento del módulo)

1. **Tema hummingbird y los botones de cantidad del carrito**. El módulo engancha el identificador del campo a las peticiones AJAX con `$(document).ajaxSend` de jQuery, y hummingbird hace esas peticiones sin jQuery. Resultado comprobado en la 9.2: el

- + añade una unidad sin personalizar como línea aparte y el `-` (que hummingbird convierte en *delete-from-cart* cuando la línea muestra una unidad) borra la línea entera con sus campos. Arreglarlo obliga a reescribir el enganche y a revisar toda la lógica de cantidades del override, que es la que calcula lo que paga el cliente. **En temas basados en classic funciona correctamente.**
- 2. **Las imágenes de los valores viven dentro del módulo** (`views/img/`): actualizar por ZIP las borra. Lo correcto sería llevarlas a `_PS_IMG_DIR`, pero eso cambia rutas ya guardadas en la base de datos de tiendas en producción.
- 3. **La opción *Product added to cart modal* escribe dentro del tema classic** (`themes/classic/modules/ps_shoppingcart/modal.tpl`, con copia en `modal-original.tpl`). Es del módulo original; no usarla en tiendas con otro tema.
- 4. **Sistema de traducción antiguo.** El módulo declara `{1 s='...' mod='an_productfields82'}` y un `translations/es.php` vacío; los textos del back-office salen en inglés. Migrarlo al sistema nuevo (`.xlf`) toca las 300 cadenas del módulo.
- 5. **`an_pf_modal_type` no se guarda cuando vale 0.** La comparación `Configuration::get(...) != '0'` es falsa cuando la clave no existe, así que no llega a escribirse. Es inocuo `—0` es el comportamiento por defecto— pero está ahí.

9. Ficha técnica

Identificación

Carpeta	<code>an_productfields82</code>
Clase	<code>an_productfields82</code> (espacio de nombres global, sin <code>namespace</code>)
Versión	2.5.0 (override marcado como 2.5.2)
Autor	Anvanto
Licencia	AFL-3.0

Tablas (prefijo de la tienda; **compartidas con el módulo** `an_productfields`)

Tabla	Contenido
<code>an_productfields</code>	Definición de cada campo: código, precio, tipo, validación, orden, obligatorio, longitud máxima, grupos, categorías, regla de impuestos
<code>an_productfields_lang</code>	Nombre y valores por idioma
<code>an_productfields_values</code>	Opciones de las listas: texto, precio, enlace e imagen
<code>an_productfields_shop</code>	Qué campo se aplica a qué tienda (multitienda)
<code>an_productfields_product</code>	Asignación campo ↔ producto y su orden
<code>an_productfields_cart</code>	Lo que el cliente ha rellenado en un carrito, por <code>values_hash</code>
<code>an_productfields_cart_values</code>	Cantidad asociada a cada <code>values_hash</code>

Hooks registrados (16)

`displayOrderDetail` · `displayAdminOrderTabOrder` · `displayAdminOrderContentOrder` · `displayAdminOrderTabLink` ·
`displayAdminOrderTabContent` · `actionAdminControllerSetMedia` · `actionPaymentModuleProductVarTplAfter` ·
`displayAdminProductsExtra` · `displayProductAdditionalInfo` · `actionProductUpdate` · `actionCartSave` · `displayHeader` ·
`displayAdminAfterHeader` · `displayCartExtraProductActions` · `actionCartUpdateQuantityBefore` · `sendMailAlterTemplateVars`

Controladores

Tipo	Clase	Para qué
Administración	<code>AdminAnProductFields82Controller</code>	Lista y formulario de campos, panel de valores (AJAX) y ajustes generales. Pestaña <code>AdminAnProductFields82</code> , bajo <code>AdminCatalog</code>
Tienda	<code>an_productfields82opcModuleFrontController</code>	Endpoint AJAX. No tiene URL amigable: se llama como <code>index.php?fc=module&module=an_productfields82&controller=opc&action=...</code> . Acciones: <code>validateProductfields</code> , <code>calculateFullprice</code> , <code>getOrderDetails</code> , <code>removeByHash</code> , <code>updateQtyByHash</code>

Override

`override/classes/Cart.php` — instalado y retirado **a mano** por el módulo (nunca con `installOverrides()`), para poder convivir con el `an_productfields` original. Sobrescribe `getProducts`, `getOrderTotal`, `getCartRules`, `deleteProduct` y `duplicate`. Lleva la constante `AN_PRODUCTFIELDS82_OVERRIDE` como marca; el módulo comprueba en cada carga del back-office que sigue ahí y lo repone si no.

Claves de configuración (`ps_configuration`, por contexto de tienda)

`an_pf_separate` · `an_pf_display_prices` · `an_pf_include_tax` · `an_pf_include_specific` · `an_pf_override_payment_module` · `an_pf_dynamic_price` · `an_pf_text_counter` · `an_pf_js_position` · `an_pf_js_position_type` · `an_pf_js_price` · `an_pf_customvalidator` · `an_pf_modal_type` · `an_pf_header_text` (multiidioma) · `an_pf_footer_text` (multiidioma). Además, `an_pf82_override_warnings` como valor global con los avisos del override.

Ficheros de la tienda

- CSS: `views/css/front/front.css`, `views/css/front/sweet-alert.css`
- JavaScript: `views/js/front/front.js` (campos y precio en vivo), `views/js/front/order.js` (carrito), `views/js/front/order-detail.js` (detalle del pedido), `views/js/front/sweet-alert.min.js`
- Imágenes de los valores: `views/img/` (protegida con `index.php` y `.htaccess`)

Cron, permisos y registros

El módulo no tiene tareas programadas ni carpeta de registros propia. Necesita permiso de escritura en `override/classes/` (para el override) y en `modules/an_productfields82/views/img/` (para las imágenes de los valores). Los errores que registra van al log estándar de PrestaShop mediante `PrestaShopLogger`.